
TASK 4: SYSTEM MODELLING AND DESIGN

Design and Implementation of a Crowdsensed Drive Test Mobile Application

By:

Chafah Perez [SENG23SE007]

Atemnkeng Destiny [SENG23SE011]

Forsoh Jehoshaphat [SENG23SE015]

Njoya Valdo [SENG23SE006]

Lecturer: Dr Nkemenyi Valery

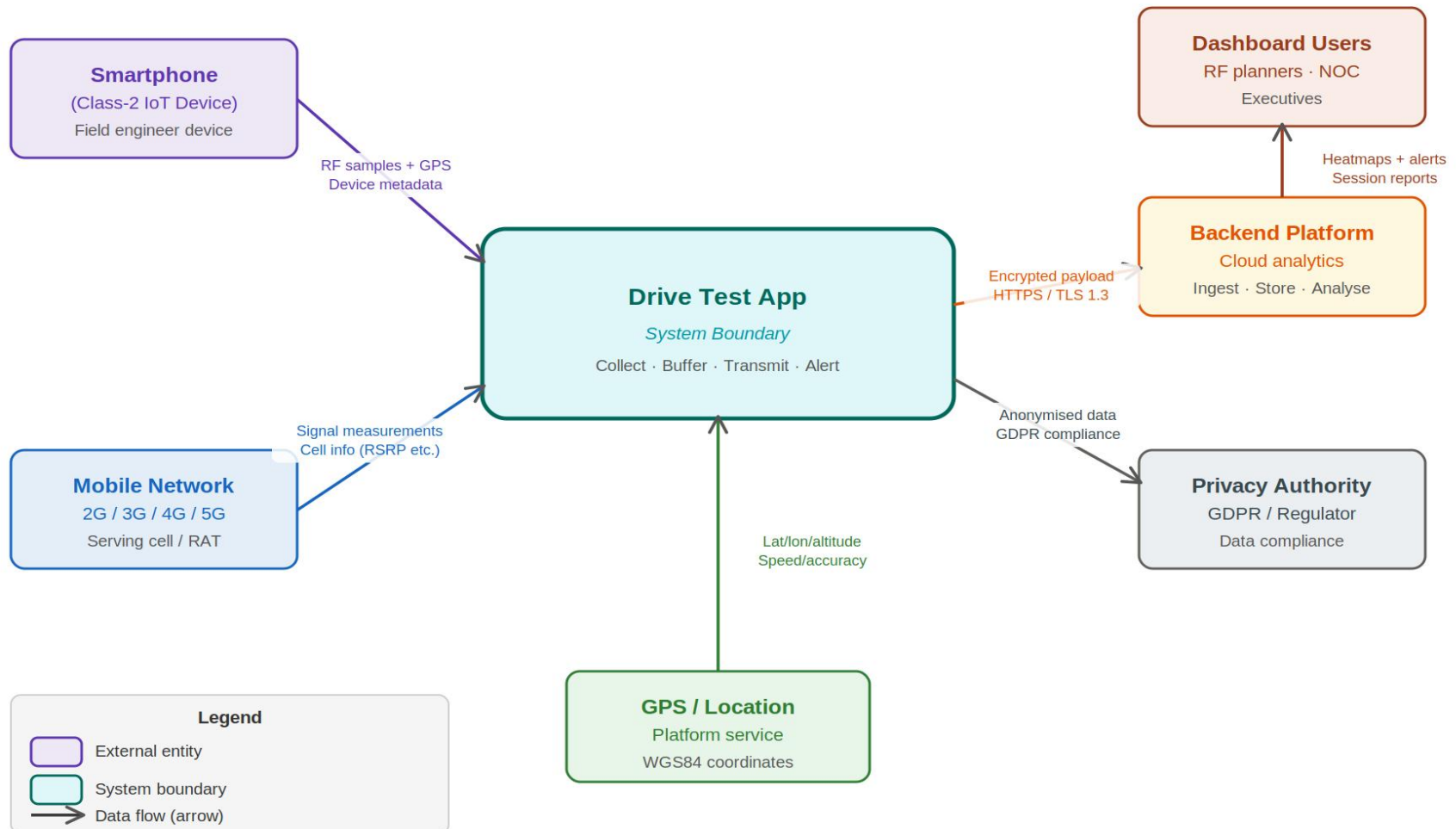
Introduction

This document presents Task 4 of the Crowdsensed Drive Test Mobile Application project System Modelling and Design. Six diagrams are produced, each providing a different perspective on the system architecture, behaviour, and deployment. Together they form a complete formal model of the system.

All diagrams use established notation standards: the Dataflow Diagram uses Yourdon-DeMarco notation (the standard for structured analysis); the Use Case, Sequence, Class, and Deployment diagrams use UML 2.5 notation as specified by the Object Management Group (OMG). The Context Diagram uses a structured free-form notation consistent with the Gane-Sarson and DeMarco traditions.

1. Context Diagram(Level 0)

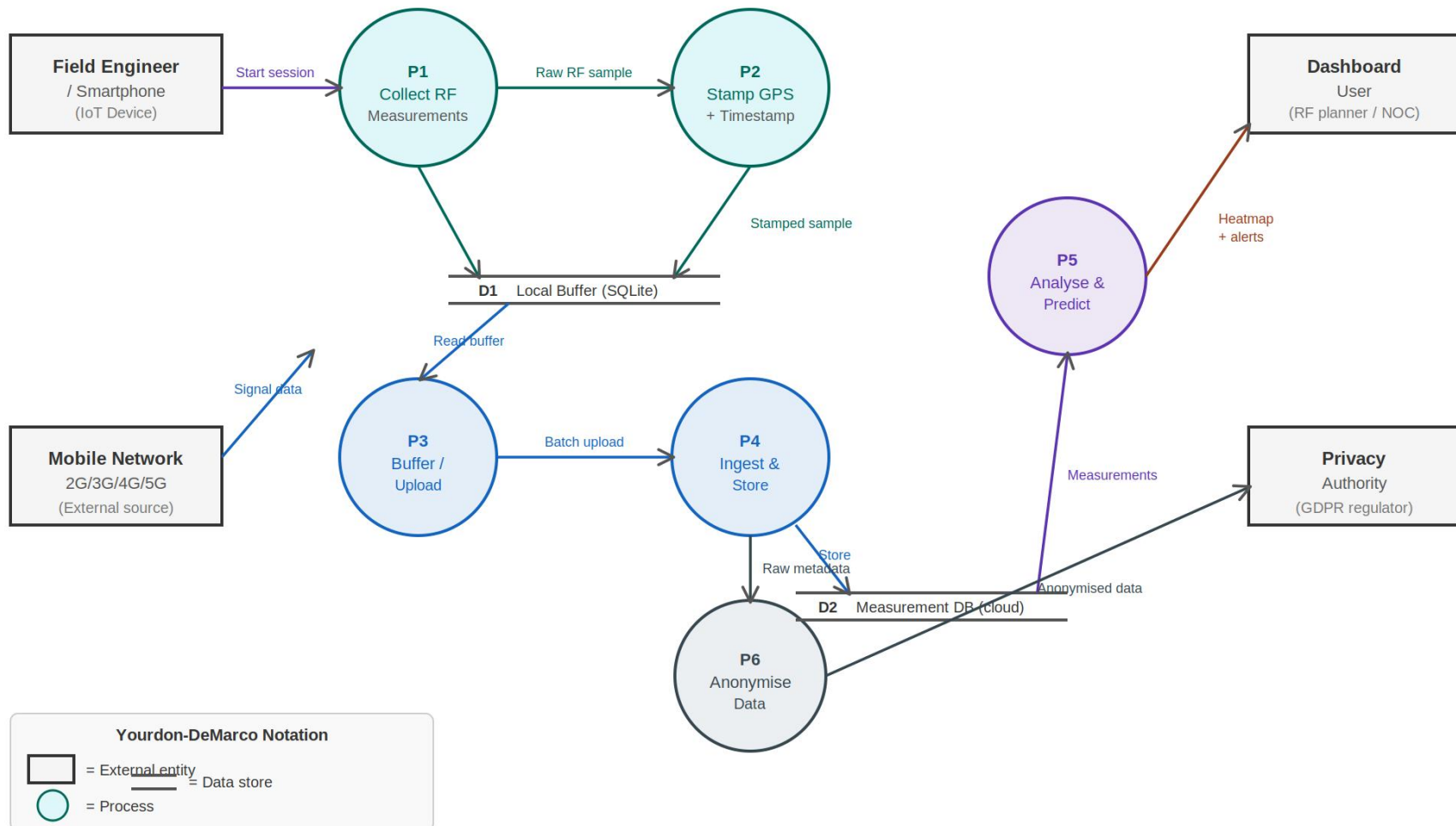
The context diagram defines the system boundary and shows all external entities that interact with the application. It is the highest-level view: the system is treated as a black box, and all data flows crossing the boundary are shown. No internal processing is depicted.



External entity	Interaction direction	Data exchanged
Smartphone (IoT Device)	Input to system	RF measurement samples, GPS coordinates, device metadata, session control events
MobileNetwork (2G/3G/4G/5G)	Input to system	Live cellular signal parameters from the serving cell and neighbour cells
Backend Platform	Output system from	Encrypted JSON measurement payloads transmitted via HTTPS with TLS 1.3
Dashboard Users	Output system from	Coverage heatmaps, coverage hole alerts, session reports, KPI dashboards
Privacy Authority	Output system from	Anonymised, de-identified measurement data for GDPR compliance
GPS / Location Service	Input to system	WGS84 latitude/longitude, altitude, GPS accuracy, speed, bearing

2. Dataflow Diagram (Level 1)

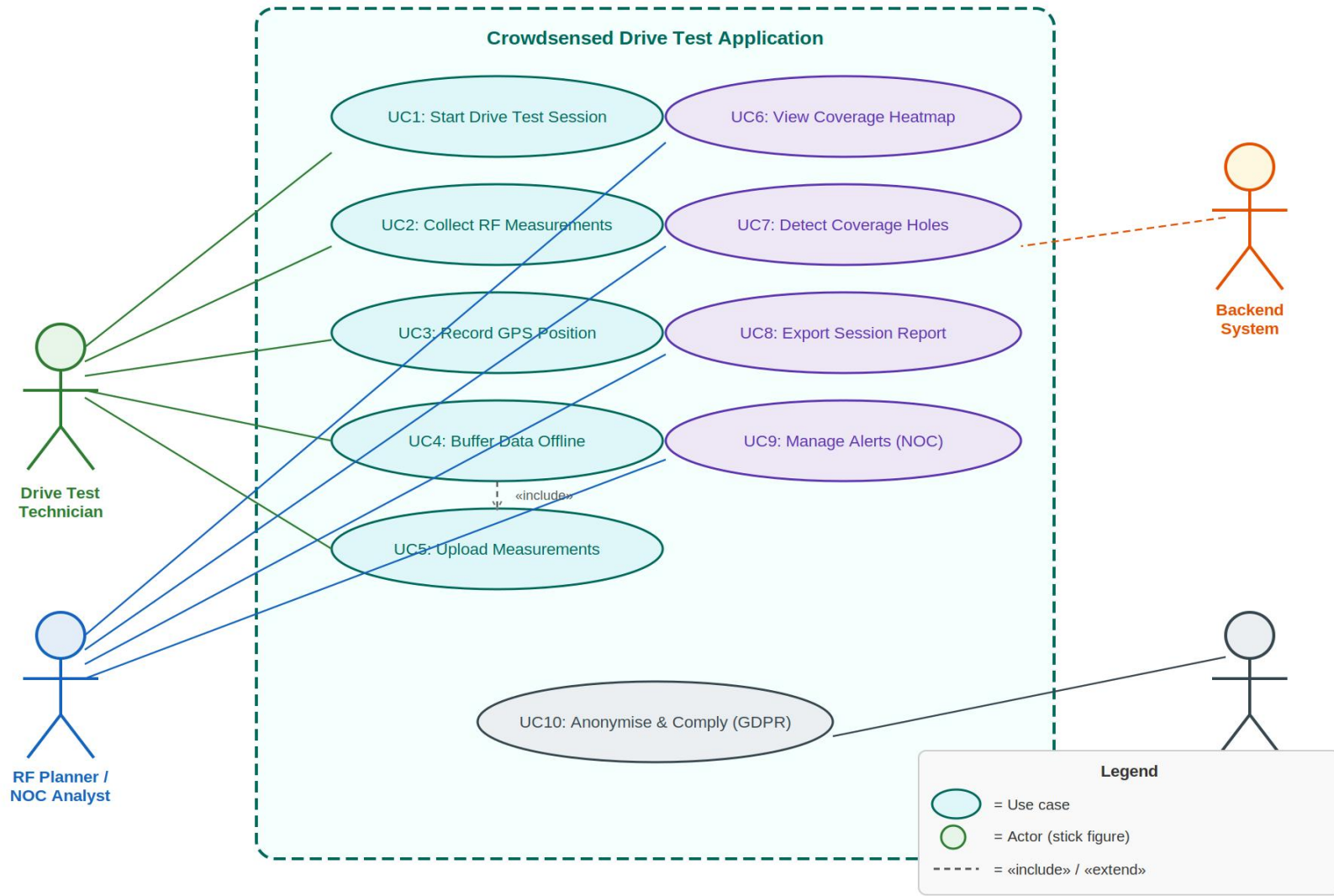
The Level 1 DFD uses Yourdon-DeMarco notation: circles represent processes, open rectangles represent external entities, parallel horizontal lines represent data stores, and arrows show data flows. It decomposes the system into six primary processes showing how data moves between them.



Process	Description	Inputs	Outputs
P1 Collect RF	Polls TelephonyManager at 1Hz	Mobile network signal data	Raw RF sample
P2 Stamp GPS	Attaches geo-context to each sample	Raw RF sample + GPS fix	Geo-stamped sample
P3 Buffer/Upload	Stores locally; uploads on connectivity	Geo-stamped sample	Batch payload
P4 Ingest & Store	Receives, validates, stores payloads	Batch payload	Stored measurement records
P5 Analyse & Predict	Interpolates heatmap; detects holes	Stored records	Heatmap + hole alerts
P6 Anonymise	Strips identifiers before compliance output	Raw metadata	Anonymised compliance data

3. Use Case Diagram

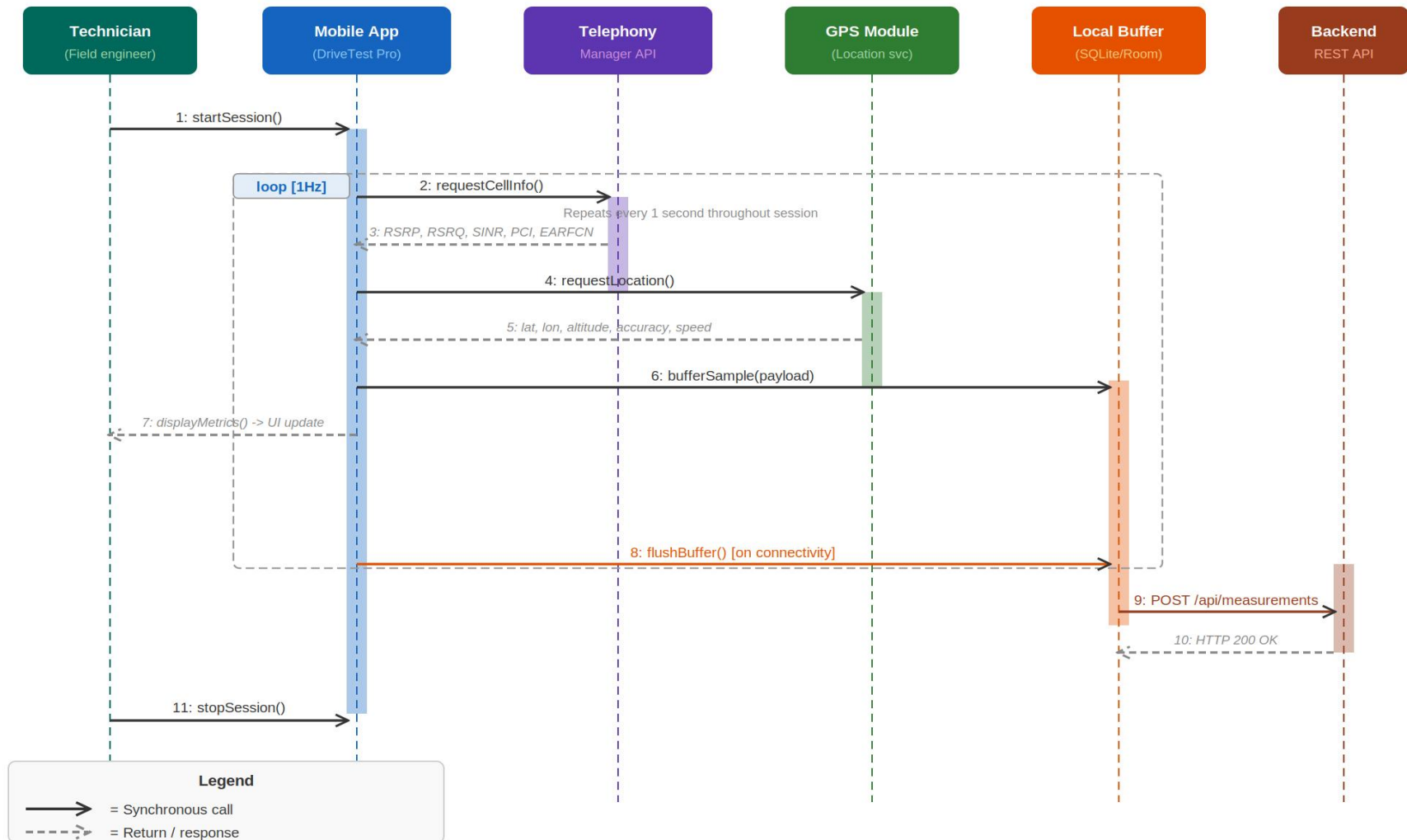
The use case diagram shows the system from the perspective of its users. Actors represent roles that interact with the system; use cases represent functions the system provides; lines show associations between actors and use cases. The dashed rectangle is the system boundary.



Actor	Type	Associated use cases
Drive Test Technician	Primary	UC1–UC5: Start session, collect RF, record GPS, buffer data, upload measurements
RF Planner / NOC Analyst	Primary	UC6–UC9: View heatmap, detect holes, export report, manage alerts
Backend System	Secondary	UC7: Coverage hole detection (automated, no human trigger)
Privacy Authority	Secondary	UC10: Anonymise and comply (GDPR)

4. Sequence Diagram

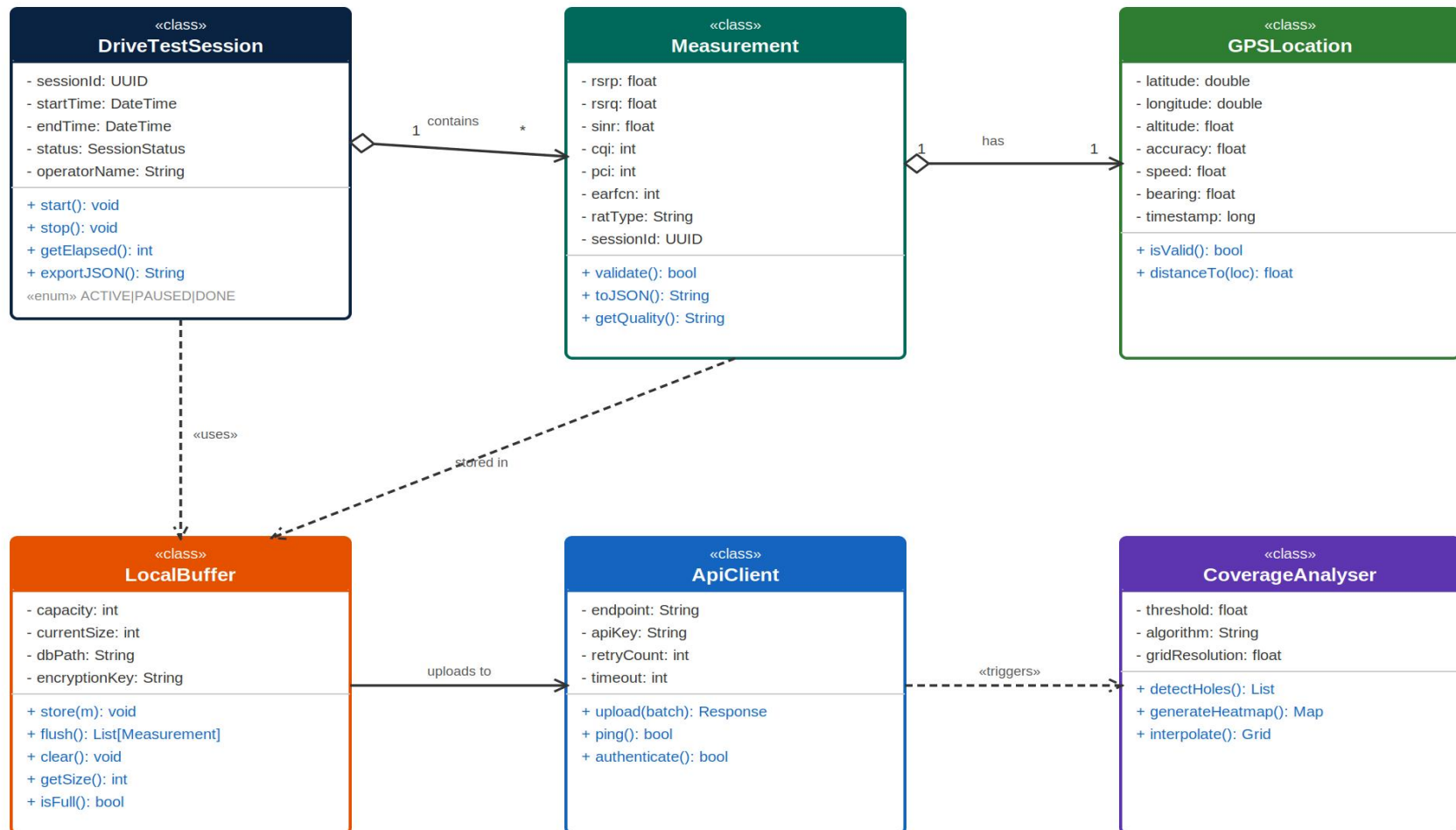
The sequence diagram shows the interaction between objects over time during a complete drive test session. Solid arrows represent synchronous calls; dashed arrows represent returns. Activation boxes show the execution period of each participant. The dashed loop box encloses the 1Hz measurement cycle.



Step	From	To	Message	Notes
1	Technician	Mobile App	startSession()	Initiates session; checks GPS fix before proceeding
2	Mobile App	TelephonyManager	requestCellInfo()	Polls RF measurements at 1Hz
3	TelephonyManager	Mobile App	RSRP, RSRQ, SINR, PCI, EARFCN	Returns current cell info
4	Mobile App	GPS Module	requestLocation()	Requests current GPS fix
5	GPS Module	Mobile App	lat, lon, altitude, accuracy, speed	Returns location data
6	Mobile App	Local Buffer	bufferSample(payload)	Stores geo-stamped sample locally
7	Mobile App	Technician	displayMetrics()	Updates UI with latest readings
8	Mobile App	Local Buffer	flushBuffer()	Triggered when connectivity available
9	Local Buffer	Backend API	POST /api/measurements	Uploads batch with TLS 1.3
10	Backend API	Local Buffer	HTTP 200 OK	Confirms receipt; buffer cleared
11	Technician	Mobile App	stopSession()	Ends session; final upload triggered

5. Class Diagram

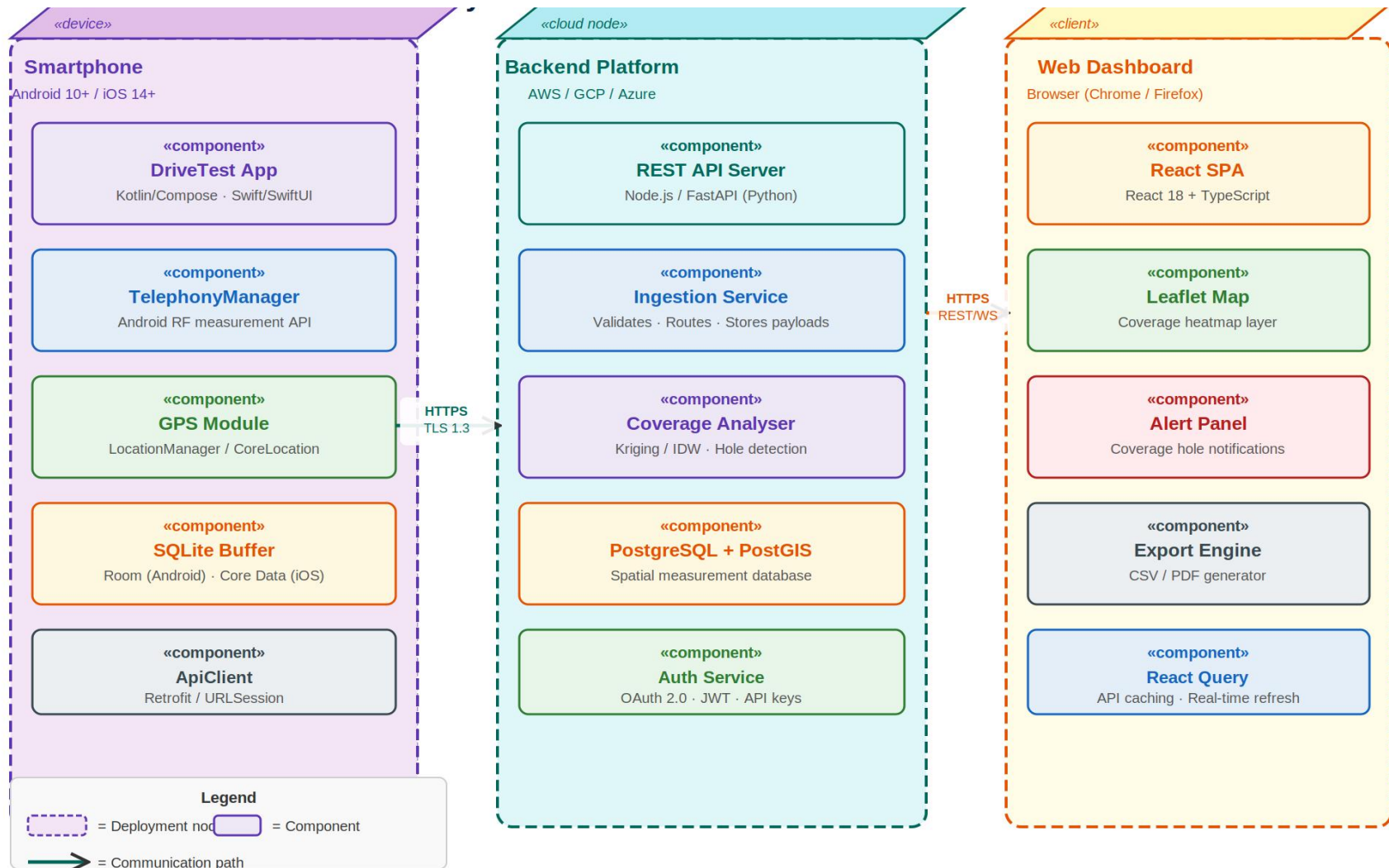
The class diagram describes the static structure of the application: the classes, their attributes and methods, and the relationships between them. Diamond ends indicate composition (strong ownership); open arrows indicate association or dependency.



Relationship	From	To	Type	Multiplicity
contains	DriveTestSession	Measurement	Composition	1 to many: session owns its measurements
has	Measurement	GPSLocation	Composition	1 to 1: every sample has exactly one GPS location
«uses»	DriveTestSession	LocalBuffer	Dependency	1 to 1: session uses buffer to persist samples
stored in	Measurement	LocalBuffer	Association	many to 1: multiple samples stored in buffer
uploads to	LocalBuffer	ApiClient	Association	1 to 1: buffer uses ApiClient to upload
«triggers»	ApiClient	CoverageAnalyser	Dependency	upload triggers backend analysis

6. Deployment Diagram

The deployment diagram shows how software components are distributed across physical or virtual hardware nodes. Dashed rectangles with 3D headers represent deployment nodes; solid rectangles inside them represent software components. Communication paths between nodes show the protocol used.



Node	Type	Components deployed	Communication
Smartphone	Physical device (Android/iOS)	DriveTest App, TelephonyManager, GPS Module, SQLite Buffer, ApiClient	HTTPS/TLS 1.3 to Backend
Backend Platform	Cloud node (AWS/GCP/Azure)	REST API Server, Ingestion Service, Coverage Analyser, PostgreSQL+PostGIS, Auth Service	HTTPS REST+WebSocket to Dashboard
Web Dashboard	Browser client	React SPA, Leaflet Map, Alert Panel, Export Engine, React Query	HTTPS REST from Backend

References

- <https://www.omg.org/spec/UML/2.5.1/>
- The Unified Modeling Language Reference Manual, 2nd ed. Addison-Wesley.
- E-UTRA Physical layer measurements
- <https://developer.android.com/reference/android/telephony/TelephonyManager>